

SK1002 · AI 辅助应用开发 · 2026-2027 学年第一学期

AI 辅助应用开发基础 (三)

Vibe Coding 工程化实践 —— 从一次任务，走向长期、稳定、可控的项目开发。

授课教师：张贤和

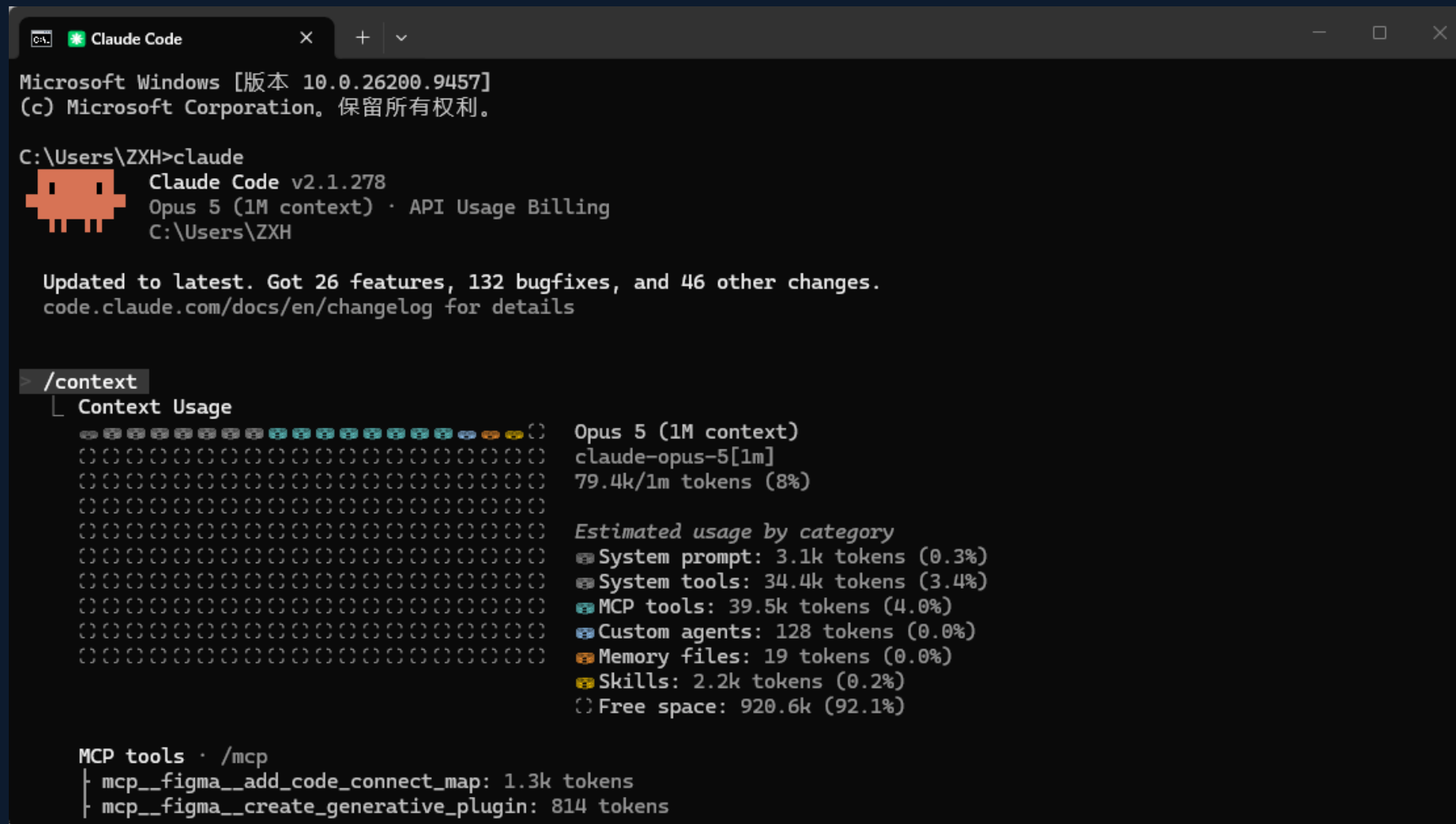
工号：3036

本讲内容

- 01 ● 管理上下文：只把当前任务需要的信息交给 AI
- 02 ● 配置项目规则：把长期有效的约束保存下来
- 03 ● 使用 Git：让每一步修改可查看、可提交、可恢复
- 04 ● 完成功能迭代：在个人作品集上形成一次真实交付

课堂主线：上下文管理 → 写入规则 → 小步修改 → 审查差异 → 提交版本 → 线上验证

为什么不能 把所有信息都丢给 AI?



信息堆积

注意力被稀释

任务边界变模糊

把上下文变成一条可控流水线。



可检查标准：AI 能说清“改哪些文件、为什么改、如何验证”，才算上下文准备完成。

AI Agent 也会先读“项目说明书”。

新会话并不等于从零开始：
规则文件会把稳定信息重新带回上下文。

OPENAI CODEX

AGENTS.md

全局：~/.codex/AGENTS.md
项目：./AGENTS.md
局部覆盖：AGENTS.override.md

越接近当前目录，优先级越高

CLAUDE CODE

CLAUDE.md

全局：~/.claude/CLAUDE.md
项目：./CLAUDE.md
模块规则：.claude/rules/*.md

每次会话开始时载入

OTHER AGENTS

Cursor

`.cursor/rules/*.mdc`

GitHub Copilot

`.github/
copilot-instructions.md`

共同点：跨任务持续存在，在 Agent 开始工作时提供稳定、可复用的上下文。

规则放在哪里，决定影响谁。

先判断作用域，再决定写入哪个文件。

01 GLOBAL · 个人默认

~/.codex/AGENTS.md

~/.claude/CLAUDE.md

适合写：语言与表达偏好、常用工具习惯、跨项目都要遵守的安全边界。

02 PROJECT · 团队共享

./AGENTS.md

./CLAUDE.md

适合写：技术栈与目录约定、构建和测试命令、修改边界、项目验收标准。

03 · LOCAL OVERRIDE

子目录规则只影响对应模块；越靠近正在处理的文件，规则越具体。

判断标准：会在多个任务中反复出现，且每次都必须遵守 → 写入规则文件。

把长期有效的要求, 写进规则。

项目开发规则

技术栈：原生 HTML / CSS / JavaScript

范围：优先复用现有结构

约束：不引入第三方 UI 库

验证：修改后必须运行并检查

兼容：桌面端与移动端都要可用

规则的价值

减少重复说明

限制修改范围

固定验证要求

让 AI 参与长期项目

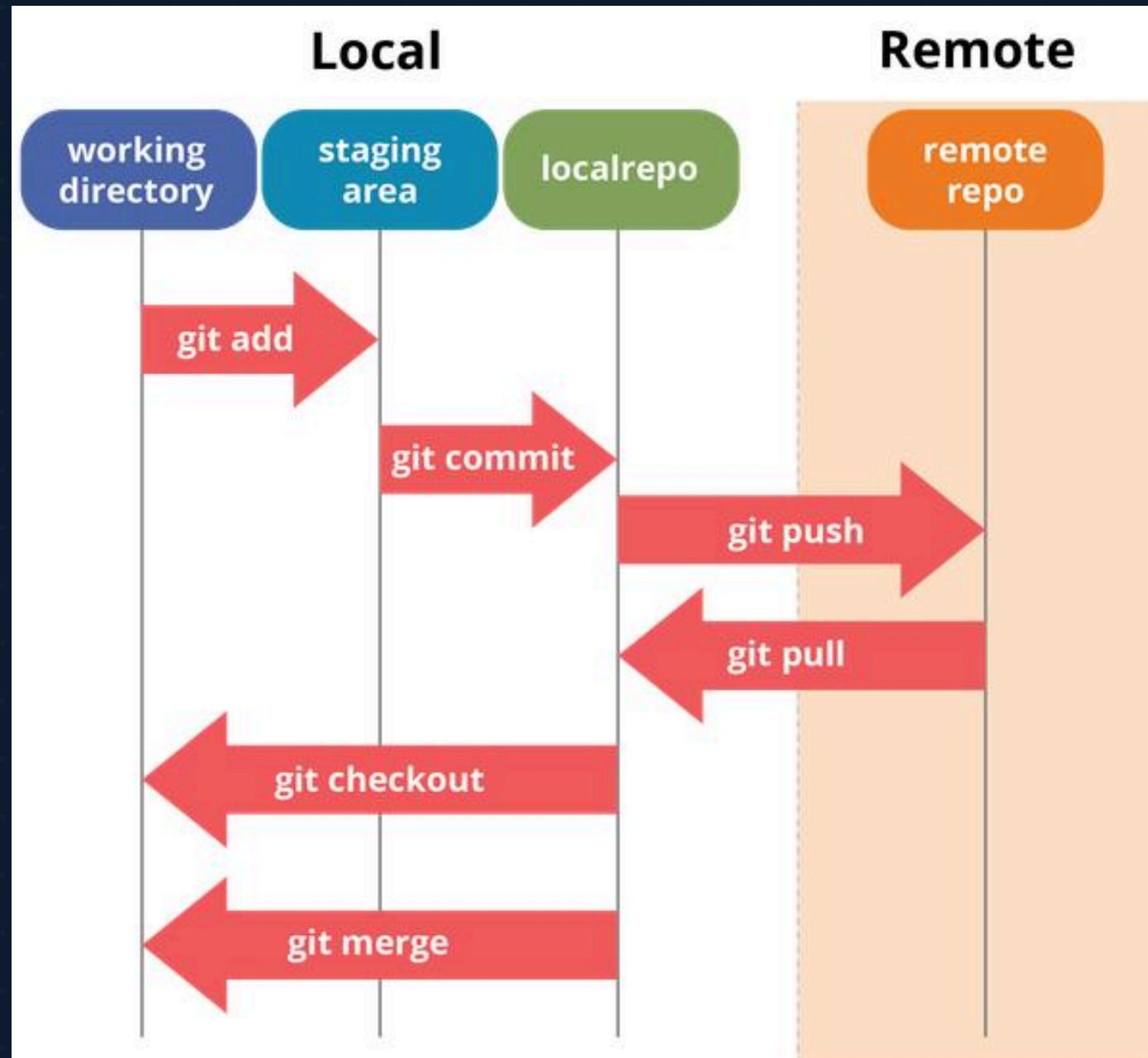


使用 Git 管理项目版本



先 status，再 add；先审查，再 commit；先本地验证，再 push。

使用 Git 管理项目版本



修改：工作区编辑文件

暂存：git add 选择要提交的修改

提交：git commit 保存本地版本

推送：git push 同步到远程仓库

拉取：git pull 获取远程更新

切换：git checkout 切换分支/恢复文件

合并：git merge 合并分支修改

实验四：在旧项目上，完成一次可追踪的迭代。

任务目标	在第 3 周个人作品集上增加 1 个功能
AI 协助点	遵守项目规则； 实现深/浅主题切换功能 辅助Git版本管理
预期输出	规则文件 + 可运行的网站 + Git 提交记录
提交材料	实验报告、截图、仓库地址

课堂优先完成：规则文件 + 1 次功能迭代 + 1 次差异审查 + 1 次有意义的提交。

让 AI 写得快， 也让项目走得稳。

上下文管理 → 项目规则 → Git 版本 → 功能迭代 → 远程部署 →
线上验证

课后：推送仓库、发布 Pages、访问线上地址并保留验证截图。

**CONTROL
THE
CHANGE**